

# Semester 2 Progress Report

Weeks 1 & 2

Multi-Agent Architecture:  
Local LLM Implementation & Boundary Testing

**Submitted to:**  
Professor Sergey Kovalchuk

**Submitted by:**  
Shahab Ali

May 8, 2026

## Contents

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>Abstract</b>                                               | <b>2</b> |
| <b>1 Week 1: Conceptual Architecture &amp; Infrastructure</b> | <b>3</b> |
| 1.1 The Three-Role Agent Taxonomy . . . . .                   | 3        |
| 1.2 Environment & Infrastructure Deployment . . . . .         | 3        |
| <b>2 Week 2: Prompt Engineering &amp; Role Testing</b>        | <b>4</b> |
| 2.1 Testing the Manager (Orchestration) . . . . .             | 4        |
| 2.2 Testing the Coder (Implementation Limitation) . . . . .   | 4        |
| <b>3 Documented AI Failures &amp; Limitations</b>             | <b>4</b> |
| 3.1 Context Collapse . . . . .                                | 4        |
| 3.2 False-Positive Safety Refusal . . . . .                   | 4        |
| 3.3 System Prompt Bleed & Execution Failure . . . . .         | 4        |
| <b>4 Conclusion &amp; Next Steps</b>                          | <b>5</b> |

## Abstract

This report details the practical implementation phase during the first two weeks of the Semester 2 project. Building upon the 6-layer taxonomy developed in Semester 1, the objective was to deploy a local 7B parameter Large Language Model (CodeLlama) within the OpenHands framework to autonomously execute specific software engineering roles. This study documents the technical infrastructure setup and presents significant experimental findings regarding the limitations of small-parameter models, specifically documenting instances of **Context Collapse**, **False-Positive Safety Refusals**, and **System Prompt Bleed** during autonomous tool execution.

## Week 1: Conceptual Architecture & Infrastructure

### The Three-Role Agent Taxonomy

To facilitate an autonomous development loop without human intervention, a three-role taxonomy was designed based on previous literature reviews:

- **Manager (Orchestration Layer):** Parses user intent into a pure logic plan without generating code.
- **Coder (Implementation Layer):** Translates the logic plan into Python and utilizes OpenHands file-creation tools to save the script to the workspace.
- **Tester (Quality Assurance Layer):** Invokes the terminal to execute the code, passing runtime errors back to the Coder for self-correction.

### Environment & Infrastructure Deployment

To ensure a secure testing sandbox and eliminate cloud API rate limits, the environment was hosted locally. Docker was utilized to run the OpenHands workspace, operating through Ubuntu via WSL to bypass Windows file-path formatting constraints. The system was successfully integrated with a local instance of the 7B CodeLlama model via Ollama. A network bridge (`host.docker.internal:11434`) was established to allow the containerized workspace to communicate seamlessly with the host GPU.

## Week 2: Prompt Engineering & Role Testing

With the environment established, practical testing evaluated the model's ability to adopt the defined personas using strict system prompts.

### Testing the Manager (Orchestration)

The model was given negative constraints:

*“DO NOT write code. Output only logic steps.”*

During initial testing, the model successfully adhered to the Orchestration persona, breaking down a string reversal task into a four-step logic plan without generating Python syntax. This confirmed that the 7B model is capable of basic task orchestration.

### Testing the Coder (Implementation Limitation)

When assigned the Coder role and instructed to physically save a file via OpenHands tools, the 7B model generated the correct syntax but fundamentally failed to invoke the necessary terminal tools. Instead of creating the file in the workspace, it defaulted to conversational output, simply typing the code into the chat interface.

## Documented AI Failures & Limitations

During rigorous stress-testing, the local 7B model exhibited severe architectural breaking points. Documenting these failures is critical to understanding the viability of small-parameter models in agentic frameworks.

### Context Collapse

When explicitly scolded for violating a negative constraint (writing code when forbidden), the model completely lost the conversational context. Instead of correcting its behavior, it hallucinated irrelevant instructions regarding GitHub Pull Requests and URL fetching. This indicates that small models struggle to maintain persona context under corrective pressure.

### False-Positive Safety Refusal

When the Coder agent was explicitly commanded to *“physically create a file on the hard drive,”* its internal safety filters triggered. The model abandoned its Coder persona and output the following refusal:

*“I cannot provide instructions on how to create a Python script named ‘reverse\_string.py’ in a specific directory. Is there anything else I can help you with?”*

This demonstrates that local models can block autonomous file creation due to over-sensitive safety filters, falsely identifying standard workspace commands as malicious system attacks.

### System Prompt Bleed & Execution Failure

The final test evaluated the QA Tester's ability to run terminal commands. The complex conditional logic caused the model to suffer from “System Prompt Bleed.” Instead of running the script, the AI regurgitated its invisible internal environment configuration, outputting text regarding the `/workspace` directory, Vite server settings, and CORS port configurations.

When the prompt was simplified to bypass this bleed, the model hallucinated a standard Bash tutorial rather than actually utilizing the OpenHands terminal to execute the Python file.

## Conclusion & Next Steps

Phase 1 testing decisively proves that while local 7B models can handle Orchestration (planning logic), they critically fail at Implementation and QA Execution due to reasoning limits, safety over-sensitivity, and tool-syntax failures.

**Strategic Pivot:** To achieve the semester goal of evaluating this architecture against the HumanEval benchmark, the project must pivot. In the upcoming weeks, I will integrate a highly capable cloud-based API (via the GitHub Student Developer Pack) into the OpenHands environment. This will resolve the tool-use limitation and allow for the extraction of quantitative benchmark data.